AI ASSISTED CODING

ASSIGNMENT-19.1

NAME : SK. Subhani

HT.NO. : 2303A52424

BATCH : 36

Lab 19 – Code Translation: Converting Between Programming Languages

Lab Objectives:

• Understand how AI tools can assist in translating code Week10 -Monday between different programming languages.

• Learn to verify correctness and functionality after translation.

• Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).

• Practice debugging and optimizing AI-translated code.

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

Expected Output:

• Equivalent working Java class translated from Python.

```python
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def get_area(self):
        return self.length * self.width

    def get_perimeter(self):
        return 2 * (self.length + self.width)

    def is_square(self):
        return self.length == self.width

print("Python Rectangle class created.")
```

```
Python Rectangle class created.
```

Now, use the following prompt with an AI assistant (like Gemini) to convert the Python class into Java:

```
Convert the following Python class to Java:

```python
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def get_area(self):
        return self.length * self.width

    def get_perimeter(self):
        return 2 * (self.length + self.width)

    def is_square(self):
        return self.length == self.width


Once you have the Java code, you can compile and run it in a Java development environment. In the meantime, I will generate Pytho
```

```python
print("\n--- Testing Python Rectangle Class ---")
rect1 = Rectangle(5, 10)

# Test methods
print(f"Rectangle 1: Length={rect1.length}, Width={rect1.width}")
print(f"Area: {rect1.get_area()}")
print(f"Perimeter: {rect1.get_perimeter()}")
print(f"Is Square: {rect1.is_square()}")
rect2 = Rectangle(7, 7)
print(f"\nRectangle 2: Length={rect2.length}, Width={rect2.width}")
print(f"Area: {rect2.get_area()}")
print(f"Perimeter: {rect2.get_perimeter()}")
print(f"Is Square: {rect2.is_square()}")
```

```
--- Testing Python Rectangle Class ---
Rectangle 1: Length=5, Width=10
Area: 50
Perimeter: 30
Is Square: False

Rectangle 2: Length=7, Width=7
Area: 49
Perimeter: 28
Is Square: True
```

After you convert the Python class to Java using an AI assistant, you can create a Java main method to test the `Rectangle` class like this (assuming your class is named `Rectangle.java`):
```

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("\n--- Testing Java Rectangle Class ---");

        // Create a Rectangle object
        Rectangle rect1 = new Rectangle(5, 10);

        // Test methods
        System.out.println("Rectangle 1: Length=" + rect1.getLength() + ", Width=" + rect1.getWidth());
        System.out.println("Area: " + rect1.getArea());
        System.out.println("Perimeter: " + rect1.getPerimeter());
        System.out.println("Is Square: " + rect1.isSquare());

        Rectangle rect2 = new Rectangle(7, 7);
        System.out.println("\nRectangle 2: Length=" + rect2.getLength() + ", Width=" + rect2.getWidth());
        System.out.println("Area: " + rect2.getArea());
        System.out.println("Perimeter: " + rect2.getPerimeter());
        System.out.println("Is Square: " + rect2.isSquare());
    }
}
```

Compare the output of the Python code above with the output of your Java code to verify the conversion.

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

• Ask AI to translate a recursive C++ factorial function into Python.

• Validate correctness by testing with sample inputs.

• Ensure Python version follows Pythonic conventions.

Expected Output:

• A correct Python implementation equivalent to the original C++ factorial function.

For this task, convert the following C++ factorial function into Python using an AI assistant (like Gemini):

```cpp
# include <iostream>

// Function to calculate factorial recursively
long long factorial(int n) {
    if (n < 0) {
        // Factorial is not defined for negative numbers
        return -1; // Or throw an exception
    } else if (n == 0 || n == 1) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

int main() {
    int num = 5;
    std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl;

    num = 0;
    std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl;

    num = 10;
    std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl;

    num = -3;
    std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl;

    return 0;
}
```

Once you have the Python function, you can use the following Python code to test it. Make sure your Python function is named `factorial`.

```python
def factorial(n):
    if n < 0:
        return -1
    elif n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
print("\n--- Testing Python Factorial Function ---")
print(f"Factorial of 5 is {factorial(5)}")
print(f"Factorial of 0 is {factorial(0)}")
print(f"Factorial of 10 is {factorial(10)}")
print(f"Factorial of -3 is {factorial(-3)}")
```

```
--- Testing Python Factorial Function ---
Factorial of 5 is 120
Factorial of 0 is 1
Factorial of 10 is 3628800
Factorial of -3 is -1
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in

Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary

FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

For this task, convert the following SQL query into a Pandas DataFrame operation using an AI assistant (like Gemini):

```
SELECT name, salary FROM employees WHERE salary > 50000 AND department = 'Sales' ORDER BY salary DESC;
```

Once you have the Python Pandas code, you can use the following Python code to test it. Make sure your Pandas code results in a DataFrame named `filtered_employees`.

```
import pandas as pd
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace'],
    'department': ['Sales', 'Marketing', 'Sales', 'IT', 'Marketing', 'Sales', 'IT'],
    'salary': [60000, 45000, 75000, 55000, 40000, 80000, 62000]
}
employees_df = pd.DataFrame(data)
print("--- Original Employees DataFrame ---")
display(employees_df)
filtered_employees = employees_df[
    (employees_df['salary'] > 50000) & (employees_df['department'] == 'Sales')
].sort_values(by='salary', ascending=False)[['name', 'salary']]
print("\n--- Filtered Employees DataFrame (Pandas Result) ---")
display(filtered_employees)
```

```
--- Original Employees DataFrame ---
```

	name	department	salary
0	Alice	Sales	60000
1	Bob	Marketing	45000

```
--- Original Employees DataFrame ---
```

	name	department	salary
0	Alice	Sales	60000
1	Bob	Marketing	45000
2	Charlie	Sales	75000
3	David	IT	55000
4	Eve	Marketing	40000
5	Frank	Sales	80000
6	Grace	IT	62000

```
--- Filtered Employees DataFrame (Pandas Result) ---
```

	name	salary
5	Frank	80000
2	Charlie	75000
0	Alice	60000

Next steps: Generate code with employees_df | New interactive sheet | Generate code with filtered_employees | New interactive sheet

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

• Prompt AI to translate code between two chosen languages.

• Test both implementations to ensure they produce the same results.

• Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

• Working equivalent code snippets in at least two different programming languages.

```python
def process_numbers(numbers):
    processed_list = []
    for num in numbers:
        if num % 2 == 0:
            processed_list.append(num * num)
    return processed_list

print("Python `process_numbers` function created.")
```

```
Python `process_numbers` function created.
```

Now, use the following prompt with an AI assistant (like Gemini) to convert the Python function into JavaScript:

````
Convert the following Python function to JavaScript:

```python
def process_numbers(numbers):
    """
    Filters even numbers from a list, squares them, and returns the new list.
    """
    processed_list = []
    for num in numbers:
        if num % 2 == 0:
            processed_list.append(num * num)
    return processed_list
````
````

```
Once you have the JavaScript code, you can test it in a JavaScript environment (e.g., a browser console, Node.js). In the meantim
```

```python
print("\n--- Testing Python `process_numbers` function ---")

# Test cases
list1 = [1, 2, 3, 4, 5, 6]
result1 = process_numbers(list1)
print(f"Original list: {list1}")
print(f"Processed list: {result1}")
list2 = [7, 8, 9, 10]
result2 = process_numbers(list2)
print(f"\nOriginal list: {list2}")
print(f"Processed list: {result2}")
list3 = [1, 3, 5]
result3 = process_numbers(list3)
print(f"\nOriginal list: {list3}")
print(f"Processed list: {result3}")
```

```
--- Testing Python `process_numbers` function ---
Original list: [1, 2, 3, 4, 5, 6]
Processed list: [4, 16, 36]

Original list: [7, 8, 9, 10]
Processed list: [64, 100]

Original list: [1, 3, 5]
Processed list: []
```

After converting the Python function to JavaScript, you can test the JavaScript `processNumbers` function (or whatever name the AI assistant gives it) in a JavaScript environment. Here's how you might test it and the expected output:

```javascript
// Assuming your converted function is named processNumbers

console.log("--- Testing JavaScript processNumbers function ---");

// Test cases
let list1 = [1, 2, 3, 4, 5, 6];
let result1 = processNumbers(list1);
console.log(`Original list: [${list1}]`);
console.log(`Processed list: [${result1}]`); // Expected: [4, 16, 36]

let list2 = [7, 8, 9, 10];
let result2 = processNumbers(list2);
console.log(`\nOriginal list: [${list2}]`);
console.log(`Processed list: [${result2}]`); // Expected: [64, 100]

let list3 = [1, 3, 5];
let result3 = processNumbers(list3);
console.log(`\nOriginal list: [${list3}]`);
console.log(`Processed list: [${result3}]`); // Expected: []
```

Challenges and Documentation: As you perform the conversion and testing, pay attention to any differences in syntax, data types (e.g., integers vs. floats, lists vs. arrays), or common idioms between Python and JavaScript. Document these challenges or interesting observations during the translation process. This helps in understanding the nuances of multi-language development.